

LLMOrchestrator

LLMOrchestrator

모든 헤드리스 CLI 코딩 에이전트를 위한 단일 제어 플레인

요약

LLMOrchestrator는 독립 실행형 재사용 가능한 Go 모듈로, 헤드리스 CLI 에이전트(OpenCode, Claude Code, Gemini, Junie, Qwen Code)를 하이브리드 파이프+파일 프로토콜로 생성, 관리, 통신합니다. 에이전트별 회로 차단기, 플러그형 멀티 프로바이더 선택, 분리된 i18n 추상화, 블러핑 방지 테스트 보장 기능을 제공합니다.

간략 설명

재사용 가능한 Go 모듈로, 하이브리드 파이프+파일 프로토콜을 통해 여러 LLM 기반 CLI 에이전트를 생성하고 제어할 수 있는 통합 인터페이스를 제공합니다. 스레드 안전 에이전트 풀링, 회로 차단기, 선택 가능한 라우팅 전략을 지원하며, 의도적으로 소비자 중립적으로 설계되었고 플러그형 i18n 번역기를 탑재했습니다.

상세 설명

LLMOrchestrator는 헤드리스 CLI 코딩 에이전트를 오케스트레이션하기 위한 공유 인프라로, 모든 멀티 에이전트 시스템이 조용히 필요로 하지만 대부분 제대로 구현하지 못하는 기반 구조입니다. OpenCode, Claude Code, Gemini CLI, Junie, Qwen Code와 같은 도구에 대해 각 프로젝트마다 프로세스 생성, 메시지 프레이밍, 결과 파싱을 재구현하는 대신, 통합된 Agent 인터페이스, 스레드 안전 AgentPool, 여러 프로바이더의 에이전트를 단일 파사드로 통합하는 MultiProviderPool을 제공합니다. 라우팅은 AgentSelector를 통해 플러그형으로 구현되며, 요구 사항을 충족하지 못하는 프로바이더를 건너뛰는 라운드 로빈 방식이나 대체 프로바이더를 활용하는 우선순위 기반 방식 등 작업 분배 정책을 선택할 수 있습니다. 각 구체적인 에이전트는 공유 BaseAdapter를 기반으로 한 얇은 어댑터로, 파이프 설

정부터 우아한 SIGTERM-then-SIGKILL 종료, 재시작, 활성 상태 확인까지 전체 프로세스 라이프사이클을 관리합니다. 즉, 번거롭고 오류가 발생하기 쉬운 부분을 한 번에 해결합니다.

통신은 의도적으로 하이브리드 방식으로 설계되어 작업에 맞는 전송 방식을 사용합니다. 파이프 전송은 빠른 대화형 메시징을 위해 개행으로 구분된 JSON를 전송하며, 요청별 읽기 데드라인과 응답 길이 제한을 적용합니다. 반면 파일 전송은 파이프에 저장하기 어려운 대용량 또는 영구적 아티팩트를 위해 세션별 인박스/아웃박스/공유 디렉터리를 사용합니다. 회복력은 사후 고려 사항이 아닌 구조적 요소로, 각 에이전트에는 연속 3회 실패 시 60초간 회로 차단기가 작동하며, 이후 반개방 상태에서 프로브를 시도합니다. 또한 백그라운드 상태 모니터가 에이전트에 핑을 보내 다운된 에이전트가 수신 트래픽을 기다리지 않고도 복구될 수 있도록 합니다. 폴 획득은 CPU를 소모하는 바쁜 대기 대신 조건 변수를 통해 블로킹되며, 응답 파서는 상태를 유지하지 않아 동시 호출이 안전합니다. 이 모듈은 철저히 분리되어 있어 소비자 관련 세부 사항이 유입되지 않으며, 모든 사용자 대상 문자열은 플러그형 i18n Translator를 거칩니다. NoopTranslator는 메시지 ID를 그대로 반환하여 누락된 번역이 눈에 띄도록 합니다.

개발 배경

모든 멀티 에이전트 시스템은 CLI 에이전트를 안정적으로 실행하고 통신해야 합니다. 프로젝트마다 프로세스 생성, 프레임링, 파싱, 장애 처리를 재구현하는 것은 비효율적이고 오류가 발생하기 쉽습니다. LLMOrchestrator는 이를 하나의 분리되고 재사용 가능한 모듈로 중앙 집중화하여, 특화된 책임 덕분에 재사용성이 보장됩니다. 소비자의 세부 사항이 유입되는 순간 재사용성은 사라집니다.

왜 이것이 게임 체인저인가

“이질적인 CLI 에이전트 군단을 운영한다”는 작업을 프로젝트별 맞춤 엔지니어링의 고된 작업에서 단 하나의 라이브러리 임포트로 전환합니다. 폴링, 서킷 브레이킹, 라이프사이클 관리, 플러그형 라우팅까지 이미 해결되고 검증된 상태입니다. 게다가 “컴파일만 되면 된다”는 식의 안일한 접근 대신 실제 시스템을 엔드투엔드로 테스트하는 안티 블러핑 테스트 덕분에, 동시성과 장애 상황에서도 실제로 신뢰할 수 있는 추상화를 얻게 됩니다. 다이어그램에서만 그럴듯해 보이는 추상화가 아닙니다.

혁신적인 점

- 하이브리드 파이프+파일 프로토콜 — 대화형 속도(JSON 라인 stdin/stdout 전송, 읽기 데드라인, 응답 크기 제한)와 내구성 있는 파일 기반 교환(수신함/발신함/공유 폴더)을 동시에 제공하여, 지연 시간과 내구성 사이에서 타협할 필요가 없습니다.
- 플러그형 셀렉터를 갖춘 멀티 프로바이더 풀 — 여러 CLI 프로바이더를 하나의 파사드로 통합하며, 라운드 로빈이나 우선순위 기반 라우팅을 정책에 따라 선택할 수 있어 하드코딩된 방식이 아닙니다.

- 에이전트별 서킷 브레이커 + 백그라운드 상태 모니터 — 자동으로 성능 저하 및 복구(3회 실패 → 60초 차단 → 반개방 프로브)를 수행하여 불안정한 에이전트를 격리하고, 수동 개입 없이 조용히 복구합니다.
- 비지 웨이트 풀링 — Acquire는 sync.Cond를 사용하여 매칭되고 건강한 에이전트가 해제되거나 컨텍스트가 취소될 때까지 블로킹되며, 대기 중 CPU를 소모하지 않습니다.
- 엄격한 디커풀링 + 안티 블러핑 국제화 — NoopTranslator는 메시지 ID를 그대로 반환하여 번역 누락이 눈에 띄도록 하며, 공백으로 처리되어 발견되지 않는 상황을 방지합니다.
- 보안 기본 설정 — 바이너리 경로 허용 목록을 통해 셸 인터플레이션을 차단하여 커맨드 인젝션 공격 표면을 없앴으며, 경로 순회 보호, 1MiB 응답 크기 제한(무한 출력 방지), 로그 내 API 키 마스킹을 지원합니다.
- 안티 블러핑 챌린지 하네스 — 실제 디스크/JSON/파서 라운드트립을 5개 언어로 테스트하며, 기능이 손상되면 반드시 비정상 종료해야 하는 뮤테이션 게이트와 쌍을 이룹니다. 이는 기능이 실제로 실패할 수 있음을 증명하는 테스트입니다.

가장 큰 기술적 도전 과제와 해결 방법

- 신뢰할 수 있는 에이전트 프로세스 I/O — 스폰된 CLI 프로세스와 통신하는 것은 겉보기보다 어렵습니다. 하이브리드 파이프+파일 전송, 메시지/파서 계약으로 양측이 와이어 포맷에 동의하도록 하고, BaseAdapter로 전체 프로세스 라이프사이클을 중앙화하여 SIGTERM 타임아웃을 우아하게 처리하고, 최종적으로 SIGKILL 폴백까지 지원합니다.
- 비지 웨이팅 없는 동시성 — AgentPool에서 뮤텍스와 조건 변수를 사용하여 Acquire가 실제로 매칭된 에이전트가 해제될 때까지 대기하도록 하고, 상태가 없고 부작용이 없는 파서를 사용하여 여러 고루틴에서 안전하게 호출할 수 있도록 해결했습니다.
- 프로바이더 장애 격리 — 하나의 불량 프로바이더가 전체 시스템을 다운시키지 않도록 에이전트별 서킷 브레이커로 영향 범위를 제한하고, 요청이 없어도 복구를 유도하는 헬스 모니터 고루틴을 통해 해결했습니다.
- 컴파일만으로는 부족한 정확성 증명 — 챌린지 러너로 en/sr/ja/es/de 5개 언어에 걸친 수십 가지 불변성을 실제 시스템에서 테스트하며, 뮤테이션 게이트(LLMORCH_MUTATE_RUNNER=1가 실패해야 → 래퍼 종료 코드 99)를 통해 기능을 의도적으로 손상시켜 게이트 자체가 허점이 아님을 증명합니다.
- 무언의 실패 없는 국제화 — NoopTranslator의 메시지 ID 원문 반환 방식과 소비자별 번역기 주입으로 번역 누락이 항상 눈에 띄도록 하여, 누락을 은폐하지 않고 해결했습니다.

콘텐츠

기술 스택

- **Go (1.25)** — 실시간 에이전트 프로세스 오케스트레이션에 필수적인 최고 수준의 동시성 처리와 깔끔한 프로세스 제어를 제공한다는 점에서 선택되었습니다. 이 모듈은 에이전트 어댑터, 전송 계층, 파서를 직접 구현합니다.

- **Go** 표준 라이브러리만 사용 (+ **testify, yaml.v3**) — 의존성 범위를 최소화하고 어떠한 LLM SDK도 포함하지 않음으로써 모듈이 가볍고 어떤 소비자 환경에도 임베드될 수 있도록 설계된 의도적인 선택입니다.
- 파이프 전송 (**JSON-lines over stdio**) — 빠른 상호작용 메시징을 위해 선택되었으며, 읽기 타임아웃과 응답 길이 제한을 통해 중단되거나 제어 불능 상태의 에이전트가 호출자를 멈추지 않도록 강화되었습니다.
- 파일 전송 (**inbox/outbox/shared**) — 세션별로 내구성이 요구되는 대용량 아티팩트 교환에 적합하도록 선택되었으며, 이 경우 파이프는 적합하지 않습니다.
- **sync.Mutex/sync.Cond** — 바쁜 대기 없이 블로킹 방식의 공정한 에이전트 풀 획득을 구현하기 위해 선택되었습니다.
- 서킷 브레이커 + **HealthMonitor** — 단순한 장애 감지를 넘어 에이전트별 복원력 및 능동적 복구를 제공하기 위해 함께 선택되었습니다.
- **pkg/i18n Translator** — 핵심 모듈에서 소비자별 문자열을 분리하는 독립된 지역화 계층으로 선택되었습니다.
- 테스트 하네스 (**challenges/runner**) + **Makefile (test -race, fuzz, cover)** — 레이스 조건 감지 및 파서 퍼징을 포함한 적대적 환경에서의 정확성을 입증하기 위한 증거 기반 검증을 위해 선택되었습니다. 이는 단순한 가정으로 그치지 않습니다.

현황 및 투명성 안내

- **현황:** 베타. 여러 Helix/vasic 프로젝트에서 서브모듈로 사용되는 독립형 재사용 모듈입니다. 라이선스: **Apache-2.0**; GitHub 저장소는 공개되어 있습니다.
- 모델 메타데이터는 HelixQA를 통해 LLMsVerifier와 연동되며, 이 모듈은 LLMsVerifier/VisionEngine/DocProcessor를 직접 임포트하지 않습니다. 상위 애플리케이션의 CLAUDE.md(Gin/PostgreSQL 등)에 언급된 스택은 helix_code를 설명하는 것이며, 이 모듈과는 무관합니다.

우선순위 tier: Helix-주요 (LLM-인프라 클러스터 — 독립형 재사용 모듈). HelixTrack 이후 순위를 가집니다.